

CodeAtlas — Whole-App Guide

Everything you need to run, configure, and connect CodeAtlas locally and in production: stack, env vars, URL changes, Supabase Postgres, Neo4j, and deployment.

1. What the app is

CodeAtlas is a code-architecture copilot:

- **Frontend** — React + Vite + TypeScript. [@xyflow/react](#) (React Flow) renders the interactive architecture map (drag to reposition nodes, double-click a node to rename it, EXPORT downloads the modified layout JSON). Aura 1.0's animated 3D avatar uses [three](#) (raw Three.js, no react-three-fiber).
- **Backend** — FastAPI + uvicorn. Auth (email/password + GitHub OAuth), repo import/parsing, authorization/RBAC, and the **Aura 1.0** endpoint (`/api/aura/chat`) which runs a fully offline intent engine (`nlu` patterns + `scikit-learn` intent classifier + `brain.py` knowledge base answering from the real project graph). No external LLM is called.
- **Persistence** — one store facade with two interchangeable backends: JSON files under `data_base/` (default) or **PostgreSQL** (Supabase/Neon/Render Postgres) when `DATABASE_URL` is set. Rows live in JSONB (`ca_collections`, `ca_audit`, `ca_graphs`, `ca_secrets`, `ca_settings`).
- **Graph stores** — Neo4j is NOT wired into the app yet; section 6 shows how.

2. Repo map

front_end/	React app (Vite). dev server <code>http://localhost:5173</code>
src/services/api.ts	<code>API_BASE_URL = import.meta.env.VITE_API_URL 127.0.0.1:8000</code>
src/components/atlas/	graph-canvas, graph-layout, atlas-node (rename UI)
src/components/aura/	aura-bar (chat), aura-avatar-3d (Three.js avatar)
back_end/	FastAPI service. runs at <code>http://127.0.0.1:8000</code>
app/main.py	app factory, CORS, loads <code>back_end/.env</code>
app/api/auth.py	register/login/GitHub OAuth (token via <code>?token=</code>)
app/api/aura.py	<code>/api/aura/chat</code> + <code>/api/aura/status</code>
app/services/store.py	FileStore ↔ Postgres facade (<code>DATABASE_URL</code> switch)
app/services/ml/	scikit-learn intent classifier + joblib model
scripts/migrate_to_db.py	one-shot copy of <code>data_base/</code> into Postgres
data_base/	JSON files (users, sessions, projects, policies...)
render.yaml	Render blueprint (backend only)

3. Local setup

Backend (run in `back_end/`):

```
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Frontend (run in `front_end/`):

```
npm install
npm run dev      # http://localhost:5173
```

`back_end/.env` is loaded automatically at startup. `front_end/.env` is read by Vite. Never commit either file — commit the `.env.example` copies instead.

4. URLs — how to change them (dev vs production)

Variable	Read by	Purpose	Dev value	Prod value
VITE_API_URL	frontend bundle	backend base URL for every API call	http://127.0.0.1:8000	https://your-api.onrender.com
BACKEND_BASE	backend env only	builds the GitHub OAuth callback URL (<code>\${BACKEND_BASE}/api/auth/github/callback</code>)	http://localhost:8000	same as VITE_API_URL
FRONTEND_URL	backend env only	OAuth redirect target after login + CORS origin	http://localhost:5173	https://your-app.vercel.app
ALLOWED_ORIGINS	backend env only	extra comma-separated CORS origins	(unset)	e.g. other preview domains

Steps to change a URL — local

1. Edit `front_end/.env` → `VITE_API_URL=...` and **restart** `npm run dev` (Vite reads env at startup; a running dev server will not pick it up).
2. Edit `back_end/.env` → `FRONTEND_URL` / `BACKEND_BASE` and **restart** `uvicorn`.
3. If you changed the backend port/host, also update the GitHub OAuth Authorization callback URL in `https://github.com/settings/developers` (must equal `http://<host>:<port>/api/auth/github/callback`).

Steps to change a URL — production

1. Backend (Render dashboard → your `codeatlas-api` service → Environment): set `BACKEND_BASE` and `FRONTEND_URL` (and `ALLOWED_ORIGINS` if needed), then deploy/restart.
2. Frontend host (Vercel project settings → Environment Variables): set `VITE_API_URL`. **VITE vars are baked at build time** — redeploy the frontend so a new production build picks it up.
3. GitHub OAuth app: callback URL must be `https://your-api.onrender.com/api/auth/github/callback`.

5. Connecting Supabase (Postgres)

The backend uses `psycopg3` (`psycopg[binary]` + `psycopg-pool`). Supabase is just a hosted Postgres, so you only need the connection string.

5a. Create the database

1. `https://supabase.com` → New project → pick a region + DB password.
2. In the dashboard: **Project Settings** → **Database** → **Connection string** → **URI**.
Two forms exist:
 - **Direct** (port `5432`) — works everywhere; recommended for this app.
 - **Connection pooler** (port `6543`) — Supervisor; the backend holds an internal connection pool, so a session-pooler URL (port `5432` mode "Session") is safer than the transaction pooler.
3. Copy the string. The backend probes SSL automatically (`sslmode=require`, falls back to plain), so a plain URI like `postgresql://postgres.<project-ref>:<password>@aws-0-<region>.pooler.supabase.com:5432/postgres` works without extra flags.

5b. Point the backend at it

1. Edit `back_end/.env`:

```
DATABASE_URL=postgresql://postgres.<project-ref>:<password>@aws-0-  
<region>.pooler.supabase.com:5432/postgres
```

- Restart uvicorn. On boot, `store.init()` runs the DDL and creates `ca_collections`, `ca_audit`, `ca_graphs`, `ca_secrets`, `ca_settings` automatically — verify in Supabase SQL Editor:

```
select name from ca_collections limit 5;
```

- First migration (only if you had JSON data in `data_base/`): copy it into Postgres **before** real traffic arrives:

```
cd back_end && python -m scripts.migrate_to_db
```

- Keep `CODEATLAS_SECRET_KEY` — it encrypts GitHub OAuth tokens at rest. Do not clear it after data exists, or stored tokens become undecryptable.

Supabase gotchas

- The free-project instance **pauses** after ~1 week of inactivity; the first request after a pause can time out while it wakes.
- Use `DATABASE_URL` (single value), not the legacy `SUPABASE_URL` + `SUPABASE_ANON_KEY` pair — this app talks to Postgres directly (psycopg), not through the Supabase REST/JS client.
- Backup is automatic on Supabase; restore via Dashboard → Database → Backups.

6. Connecting Neo4j (graph database — optional, not wired in yet)

The app currently persists graphs as JSONB in Postgres. To add Neo4j for native graph traversal (paths, cycles, impact queries), do this:

6a. Get an instance

- AuraDB (managed, has free tier):** <https://console.neo4j.io> → Create a database → copy the connection URI + credentials.
- Local Docker:**

```
docker run -d --name codeatlas-neo4j -p 7474:7474 -p 7687:7687 \
-e NEO4J_AUTH=neo4j/<password> -v neo4j_data:/data neo4j:5
```

URI is `bolt://localhost:7687` (AuraDB uses `bolt+s://<instance>.databases.neo4j.io`). Allowlist your IP in the AuraDB dashboard if connections time out.

6b. Add the driver

```
pip install neo4j
```

Add `neo4j` to `back_end/requirements.txt`, then create `back_end/app/services/neo4j_store.py`:

```
"""Optional Neo4j graph store. Active only when NEO4J_URI is set."""
import os
from neo4j import GraphDatabase

_DRIVER = None

def _init() -> None:
```

```

global _DRIVER
uri = os.environ.get("NEO4J_URI", "").strip()
if not uri:
    return
_DRIVER = GraphDatabase.driver(
    uri,
    auth=(os.environ["NEO4J_USER"], os.environ["NEO4J_PASSWORD"]),
)

def init() -> None:
    _init()
    if _DRIVER:
        with _DRIVER.session() as s:
            s.run("RETURN 1")

def close() -> None:
    if _DRIVER:
        _DRIVER.close()

def store_graph(project_id: str, nodes: list[dict], edges: list[dict]) -> None:
    """Upsert a project's nodes/edges. Intended to be called from the
    import/parsing pipeline right after a graph is produced."""
    if not _DRIVER:
        return
    with _DRIVER.session() as s:
        for node in nodes:
            s.run(
                "MERGE (n:CodeNode {project_id: $pid, id: $id}) "
                "SET n.label = $label, n.type = $type, n.path = $path",
                pid=project_id, id=node["id"], label=node["label"],
                type=node["type"], path=node["path"],
            )
        for edge in edges:
            s.run(
                "MATCH (a:CodeNode {project_id: $pid, id: $s}), "
                "(b:CodeNode {project_id: $pid, id: $t}) "
                "MERGE (a)-[r:RELATES {relation: $relation}]->(b)",
                pid=project_id, s=edge["source"], t=edge["target"],
                relation=edge["relation"],
            )

```

Wire it into the app lifecycle in `back_end/app/main.py` (lifespan):

```

from app.services import neo4j_store
# inside lifespan, after store.init():
neo4j_store.init()
# and on shutdown:
neo4j_store.close()

```

6c. Env vars

Add to `back_end/.env` (and as secrets in Render/Supabase env when deploying):

```

NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=<password>

```

With these set (and `DATABASE_URL` also set), Postgres stays the source of truth and Neo4j mirrors the graph for path/cycle/impact queries. Leave `NEO4J_URI` unset and nothing changes.

7. Production deployment (end to end)

1. **Backend** — Render: New + → Blueprint → this repo (`render.yaml`). Set in the dashboard environment: `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`, `FRONTEND_URL`, `BACKEND_BASE`, `DATABASE_URL`. `CODEATLAS_SECRET_KEY` is auto-generated on first deploy. Health check: `GET /healthz`.
2. **Run one uvicorn process / no replicas** — sessions and the SSE event bus live in memory; multiple instances will not share them yet.
3. **Frontend** — Vercel: import `front_end/` as the root, build command `npm run build`, output `dist`, and set `VITE_API_URL` to the Render URL.
4. **GitHub OAuth** — callback URL = `https://your-api.onrender.com/api/auth/github/callback`.

8. Troubleshooting

- **ERR_NETWORK on the parsed map / login** — frontend cannot reach the backend: check `VITE_API_URL`, restart the dev server, confirm uvicorn is up with `curl http://127.0.0.1:8000/healthz`.
- **WSL can't curl the backend** — the uvicorn process runs on Windows and WSL loopback is isolated; test from PowerShell or `python.exe`, not WSL.
- **npm run dev dies with a rolldown binding error** — `@rolldown/binding-*` is platform-specific; don't run a bare `npm install` (it prunes the other platform bindings from `node_modules/@rolldown/`).
- **UnicodeEncodeError (cp1252) when printing Aura's reply in tests** — cosmetic; the API returns UTF-8 JSON, only the Windows console print fails.
- **OneDrive reverting files** — the repo sits inside OneDrive, which can silently restore older versions; commit often.
- **Supabase connection timeouts** — unpause the project, or switch to the session-pooler port (`5432`) URL.